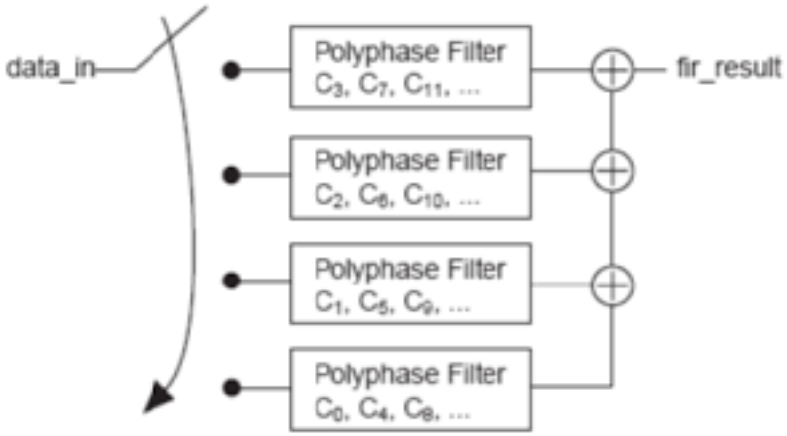
Computer Assignment 2
Nathan Walker
A02364124

Introduction:

Polyphase filtering is a process to reduce computations necessary for interpolation and decimation. This is done by splitting the interpolation/decimation filter into L or M parts or L x M for a L/M decimations.

$$H(z) = H_0(z) + z^{-1}H_1(z)$$

With the filter split like this the signal flow splits between two branches. Since one branch is delayed before the decimator. The signal is essentially commutated between to two branches, this produces efficient computations and fast computation time.



Method of Work:

Designing the filter

To design the filter for the ¾ decimation factor the correct cutoff frequency must be chosen. By design you must have the cutoff frequency be pi/max[L, M]. Using this fact the cutoff frequency was chosen to be pi/4or 1/8. I chose to have the pass band of the filter be fp = .12 and the stop band of fs = .13. Using matlabs filter designer tool with the equiripple method I chose passband ripple as Ap = .001 and stop band as As = 80 dB. To get this specifications the length of the filter is 479 taps

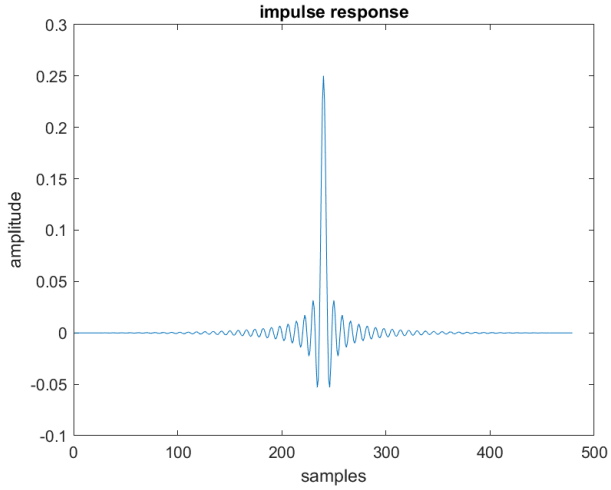


Figure 1: Impulse response of the lowpass filter for the decimator.

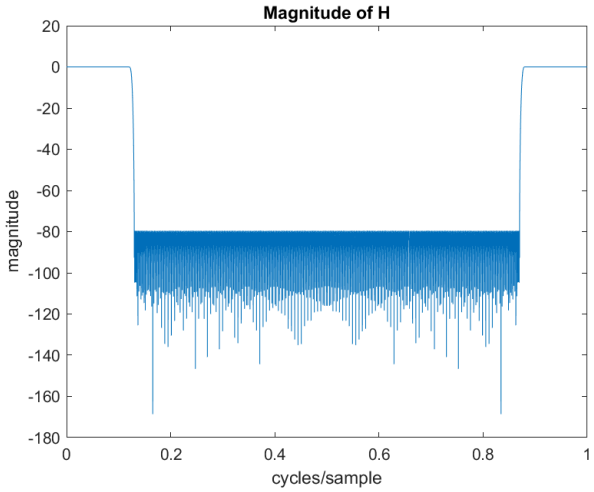


Figure 2: Magnitude(dB) of the filter showing the steep transition band and side lobe level.

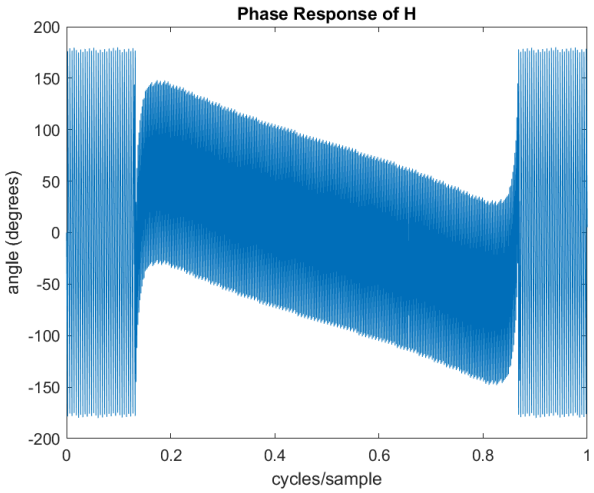


Figure 3: Wrapped phase response of the filter showing the linear phase of the filter.

Signal flow and computations

For splitting the filter and deriving the signal flow refer to the appendix for a more detailed step by step of the derivation. With the ¾ decimator and an input sampling rate of 11.025 kHz the output sample rate should be 8.269 kHz. With N = 479, L = 3, and M = 4 using equations derived in class the number of operations comes out to.

$$\frac{N+1}{M}MPU + \frac{N+1-L}{M}APU = 239.25AMPU$$

Splitting the filter

When splitting the filter I first split the filter into three separate ones. This step can be shown in the appendix in the code as well as the signal flow derivation.

$$H_l(z) = z^{-l}H(z^L)$$

Further in the signal flow derivation when I start to decompose it more to do the downsampling portion the filter is then split as such.

$$H_lm(z) = z^{-m}H_l(z^M)$$

Testing with cos

The next 6 plots will be the DFT of the input(x) and the output(y). In each case the frequency has increased by a factor of 4/3. As looking at the graphs there are side lobes on the polyphase DFT that are higher than the traditional decimator. However, looking at the signal in time it appears to be very insignificant. As well as when you use a higher number DFT(N = 2^16) the side lopes disappear and we are left with essentialy impulses.

fo	traditional	polyphase
1/16	1/12	1/12
1/8	1/6	1/6
1/4	1/3	1/3

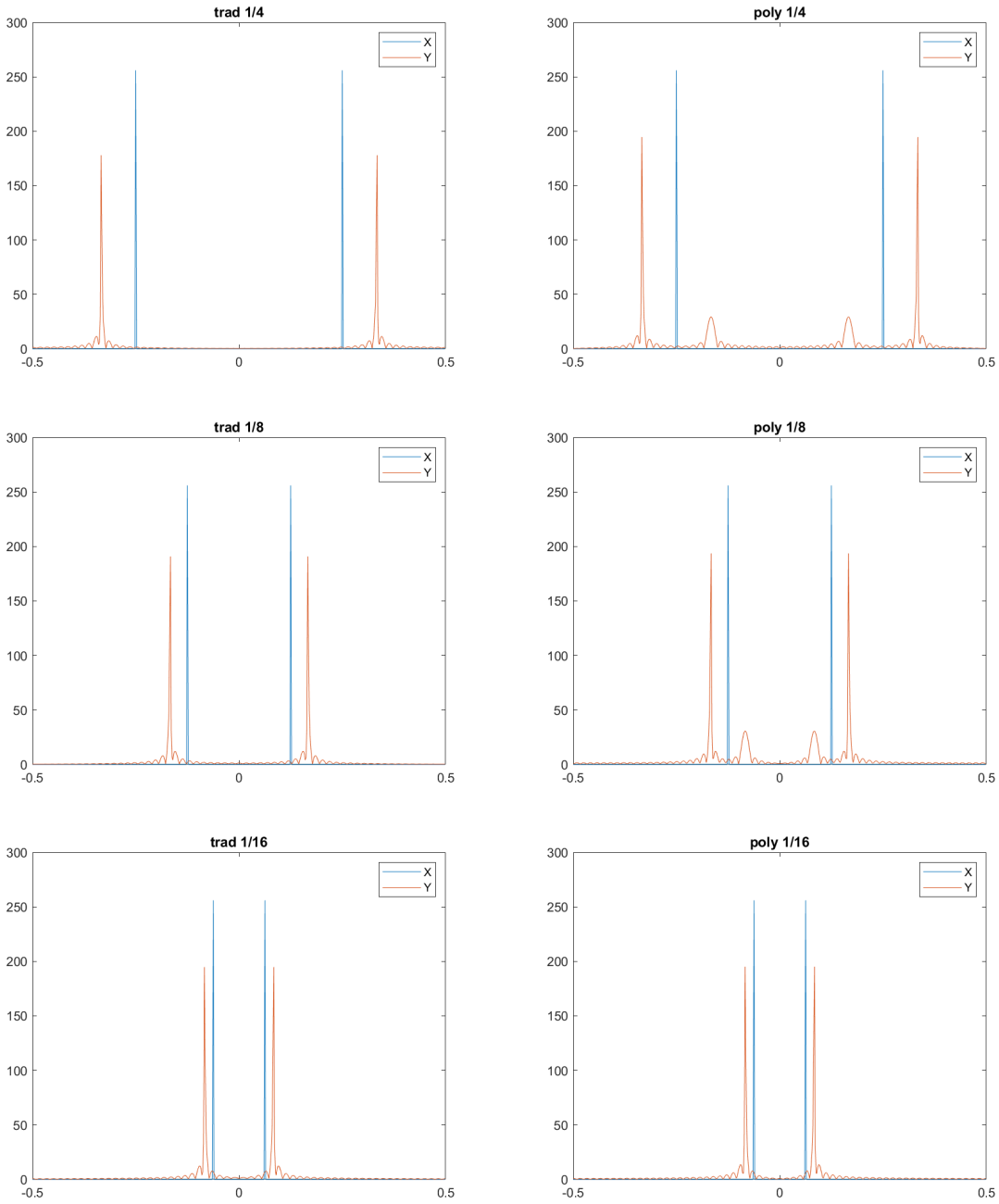


Figure 4: all the DFT plots of the traditional vs polyphase decimation.

With audio file

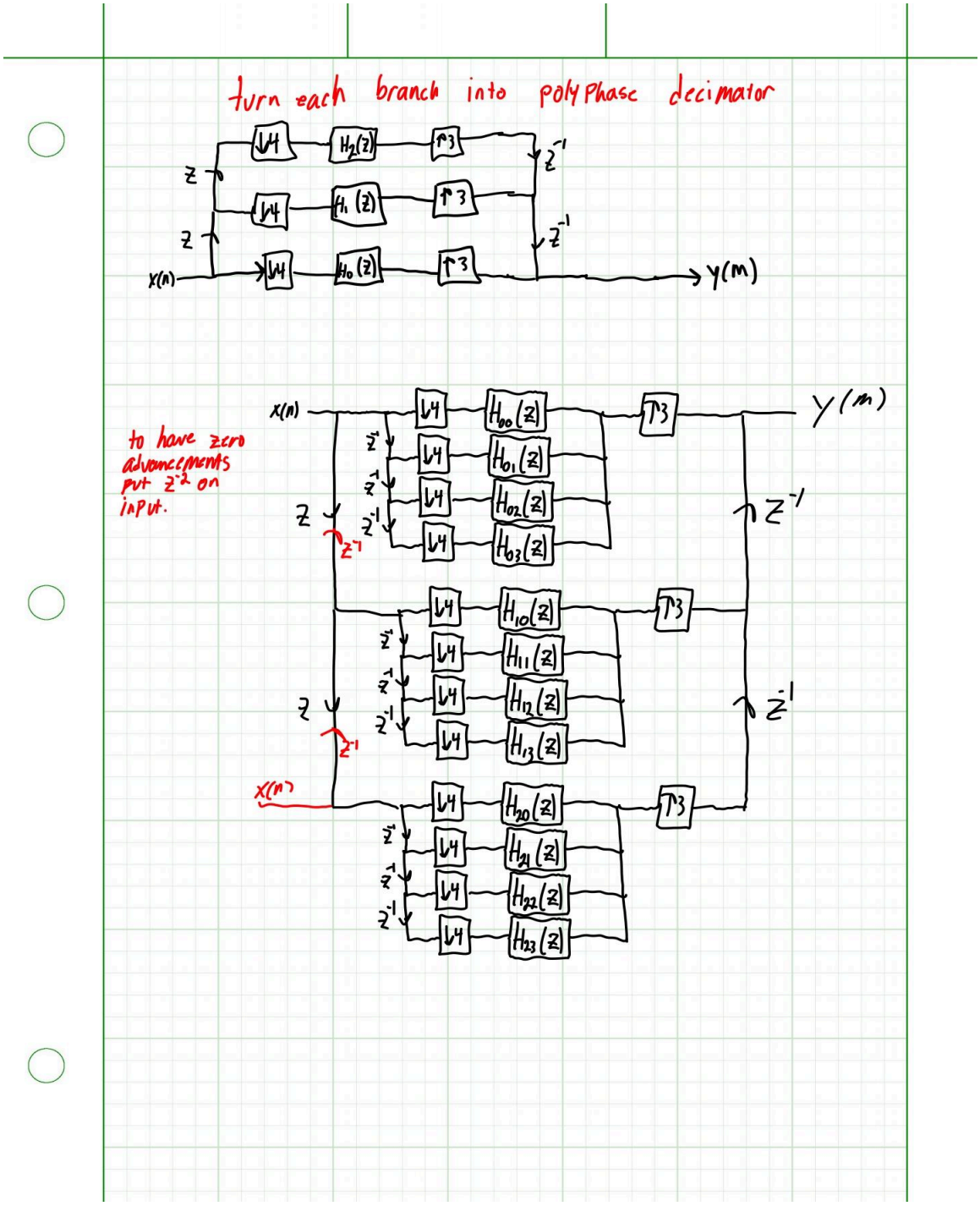
Using matlab I read the .wav files and exported them as double binary files to read into for the C program. I was initially not expecting a significant decrease in time but I was caught off guard at how much time was saved. On average the traditional downsampler took ~.5 seconds while the polyphase downsampler ~.05 seconds. Thats a huge amount of savings especially if needed for an real time application. This difference could be made bigger by seeing how wide the transition band is and reducing length of filter. However, the crucial part is after listening to both downsampled versions they sound the same. So there is significant time saved with no loss in quality. The .wav files that I produced will be attached in submission.

Conclusion:

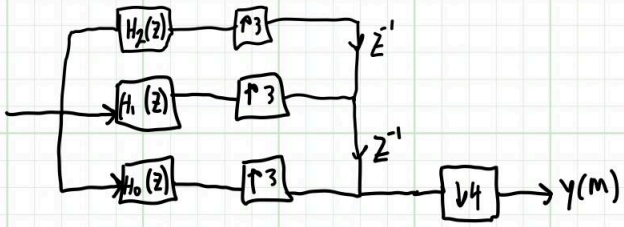
In conclusion this computer assignment showed that polyphase decimating significantly saves on time compared to traditional decimating. This can lead to many advantages in applications for real time processing or allowing for higher order filters to be used as it as time to spare. The assignment also proved that realizing systems like this is more manageable and doable than originally predicted. When looking at the signal flow chart it seems daunting but using the commutation method proves not a challenge.

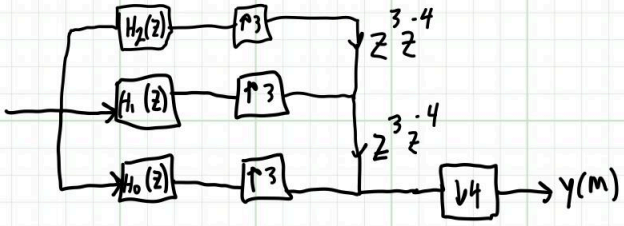
Appendix

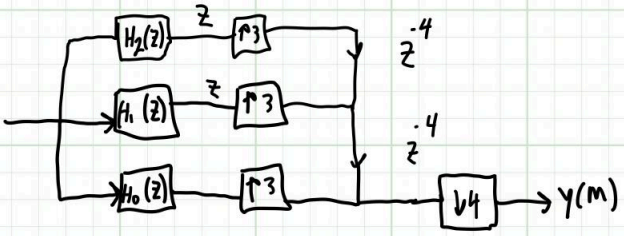
Signal Derivation

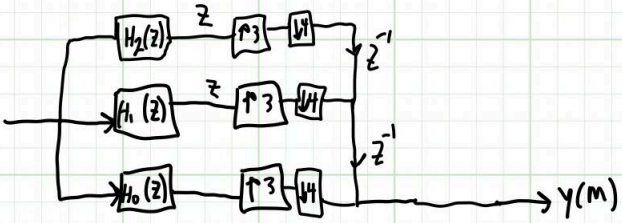



↑3
H(z)
↓4


H2(z)
H1(z)
H0(z)
↑3
z-1
↓4
y(m)


H2(z)
H1(z)
H0(z)
↑3
z3·4
z3·4
↓4
y(m)


H2(z)
H1(z)
H0(z)
z
↑3
z-4
z·4
↓4
y(m)


H2(z)
H1(z)
H0(z)
z
↑3
↓4
z-1
z-1
y(m)

C++ Code

```
//L/M = 3/4
// L = 3 M = 4
#include <iostream>
#include <string>
#include <cmath>
#include <fstream>
#include <vector>
#include <chrono>
#define L 3
#define M 4
#define N 479
#define PI  3.14159265358979323846

using namespace std;

void problem4(double* h);
void problem5(double* h);
void problem6(double* h);
double FIR_filtering(double input, double* filter, double*
filter_buffer, int length);
void split_filter(double* h_in, double* h_out, int in_L,
int split, int step);

int main(void) {

    double* h = new double[N];
    ifstream infile("lowpass_filter.bin", ios::binary |
ios::in);
    infile.read(reinterpret_cast<char*>(h), sizeof(double)
* N);
    //problem4(h);
    //problem5(h);
```

```
    problem6(h);

    return(0);
}

void split_filter(double* h_in, double* h_out, int in_L,
int split, int step)
{
    for(int i = 0; i < in_L/split; i++)
    {
        h_out[i] = 0;
    }
    int H_tracker = 0;
    for(int i = 0; i < in_L/split; i++)
    {
        if((i+step) < in_L)
        {
            h_out[i] = h_in[(i*split) + step];//H0 -> h
indexs: 0 3 6 9
        }
//H1 -> h indexs: 1 4 7 10
    }
//H2 -> h indexs: 2 5 8 11
}

double FIR_filtering(double input, double* filter, double*
filter_buffer, int length)
{
    double sum = 0;
    for(int i = length-1; i > 0; i--)
    {
        filter_buffer[i] = filter_buffer[i-1];
        sum += filter_buffer[i] * filter[i];
```

```
    }
    filter_buffer[0] = input;
    sum += filter_buffer[0] * filter[0];
    return(sum);
}

void upsample(double input, int factor, double* output)
{
    output[0] = input;
    for(int i = 1; i < factor; i++)
    {
        output[i] = input;
    }
}

void problem5(double* h)
{
    double* H0 = new double[160];
    double* H1 = new double[160];
    double* H2 = new double[160];
    double* H00 = new double[40];
    double* H01 = new double[40];
    double* H02 = new double[40];
    double* H03 = new double[40];
    double* H10 = new double[40];
    double* H11 = new double[40];
    double* H12 = new double[40];
    double* H13 = new double[40];
    double* H20 = new double[40];
    double* H21 = new double[40];
    double* H22 = new double[40];
    double* H23 = new double[40];
    int h_tracker = 0;
```

```
split_filter(h, H0, N, 3, 0);
split_filter(h, H1, N, 3, 1);
split_filter(h, H2, N, 3, 2);

split_filter(H0, H00, 160, 4, 0);
split_filter(H0, H01, 160, 4, 1);
split_filter(H0, H02, 160, 4, 2);
split_filter(H0, H03, 160, 4, 3);

split_filter(H1, H10, 160, 4, 0);
split_filter(H1, H11, 160, 4, 1);
split_filter(H1, H12, 160, 4, 2);
split_filter(H1, H13, 160, 4, 3);

split_filter(H2, H20, 160, 4, 0);
split_filter(H2, H21, 160, 4, 1);
split_filter(H2, H22, 160, 4, 2);
split_filter(H2, H23, 160, 4, 3);

int n = 100* N;
double** filter_buffer = new double*[12];
double f = 1.0/16;
double output_M[n * L / M];

for(int i = 0; i < 12; i++)
{
     filter_buffer[i] = new double[40];
}

int output_counter_L = 0;
int output_counter_M = 0;
int m_trck = 0;
```

```
double H0_buffer[3];
for(int i = 0; i < 3; i++)
{
     H0_buffer[i] = 0;
}
double H1_buffer[2];
for(int i = 0; i < 2; i++)
{
     H1_buffer[i] = 0;
}
double* H0_filters = new double[4];
double* H1_filters = new double[4];
double* H2_filters = new double[4];

for(int j = 0; j < 4; j++)
{
     H0_filters[j] = 0;
     H1_filters[j] = 0;
     H2_filters[j] = 0;
}

double H0_output = 0;
double H1_output = 0;
double H2_output = 0;

for(int i = 0; i < 2; i++)
{
     H1_buffer[i] = 0;
}
for(int i = 0; i < 3; i++)
{
     H0_buffer[i] = 0;
```

```
    }
    for(int i = 0; i < n; i++)
    {
        double temp = cos(2 * PI * f * i);
        for(int j = 2; j > 0; j--)
        {
            H0_buffer[j] = H0_buffer[j-1];
        }
        H0_buffer[0] = temp;
        H1_buffer[1] = H1_buffer[0];
        H1_buffer[0] = temp;

        if(i % 4 == 0 )
        {
            for(int j = 0; j < 4; j++)
            {
                H0_output += H0_filters[j];
                H1_output += H1_filters[j];
                H2_output += H2_filters[j];
            }
            output_M[output_counter_M++] = H0_output;
            output_M[output_counter_M++] = H1_output;
            output_M[output_counter_M++] = H2_output;
            H0_output = 0;
            H1_output = 0;
            H2_output = 0;
        }

        switch(i%4) {
            case 0:
                H0_filters[0] = L *
FIR_filtering(H0_buffer[2], H03, filter_buffer[11], 40);
                H1_filters[0] = L *
```

```
FIR_filtering(H1_buffer[1], H13, filter_buffer[7], 40);
                    H2_filters[0] = L * FIR_filtering(temp,
H23, filter_buffer[3], 40);
                    break;
                case 1:
                    H0_filters[1] = L *
FIR_filtering(H0_buffer[2], H02, filter_buffer[10], 40);
                    H1_filters[1] = L *
FIR_filtering(H1_buffer[1], H12, filter_buffer[6], 40);
                    H2_filters[1] = L * FIR_filtering(temp,
H22, filter_buffer[2], 40);
                    break;
                case 2:
                    H0_filters[2] = L *
FIR_filtering(H0_buffer[2], H01, filter_buffer[9], 40);
                    H1_filters[2] = L *
FIR_filtering(H1_buffer[1], H11, filter_buffer[5], 40);
                    H2_filters[2] = L * FIR_filtering(temp,
H21, filter_buffer[1], 40);
                    break;
                case 3:
                    H0_filters[3] = L *
FIR_filtering(H0_buffer[2], H00, filter_buffer[8], 40);
                    H1_filters[3] = L *
FIR_filtering(H1_buffer[1], H10, filter_buffer[4], 40);
                    H2_filters[3] = L * FIR_filtering(temp,
H20, filter_buffer[0], 40);
                    break;
            }
        }
        ofstream outfile3("outputM_cos_16_p5.bin", ios::binary
| ios::out);
       outfile3.write(reinterpret_cast<const char*>(output_M),
```

```
(n * L / M) * sizeof(double));
    outfile3.close();

     f = 1.0/8;
     output_counter_L = 0;
     output_counter_M = 0;
     m_trck = 0;
     H0_output = 0;
     H1_output = 0;
     H2_output = 0;

     for(int i = 0; i < 2; i++)
     {
          H1_buffer[i] = 0;
     }
     for(int i = 0; i < 3; i++)
     {
          H0_buffer[i] = 0;
     }

     for(int i = 0; i < n; i++)
     {
          double temp = cos(2 * PI * f * i);
          for(int j = 2; j > 0; j--)
          {
               H0_buffer[j] = H0_buffer[j-1];
          }
          H0_buffer[0] = temp;
          H1_buffer[1] = H1_buffer[0];
          H1_buffer[0] = temp;

          if(i % 4 == 0 )
          {
```

```
            for(int j = 0; j < 4; j++)
            {
                H0_output += H0_filters[j];
                H1_output += H1_filters[j];
                H2_output += H2_filters[j];
            }
            output_M[output_counter_M++] = H0_output;
            output_M[output_counter_M++] = H1_output;
            output_M[output_counter_M++] = H2_output;
            H0_output = 0;
            H1_output = 0;
            H2_output = 0;
        }

        switch(i%4) {
            case 0:
                H0_filters[0] = L *
FIR_filtering(H0_buffer[2], H03, filter_buffer[11], 40);
                H1_filters[0] = L *
FIR_filtering(H1_buffer[1], H13, filter_buffer[7], 40);
                H2_filters[0] = L * FIR_filtering(temp,
H23, filter_buffer[3], 40);
                break;
            case 1:
                H0_filters[1] = L *
FIR_filtering(H0_buffer[2], H02, filter_buffer[10], 40);
                H1_filters[1] = L *
FIR_filtering(H1_buffer[1], H12, filter_buffer[6], 40);
                H2_filters[1] = L * FIR_filtering(temp,
H22, filter_buffer[2], 40);
                break;
            case 2:
                H0_filters[2] = L *
```

```
FIR_filtering(H0_buffer[2], H01, filter_buffer[9], 40);
                    H1_filters[2] = L *
FIR_filtering(H1_buffer[1], H11, filter_buffer[5], 40);
                    H2_filters[2] = L * FIR_filtering(temp,
H21, filter_buffer[1], 40);
                    break;
                case 3:
                    H0_filters[3] = L *
FIR_filtering(H0_buffer[2], H00, filter_buffer[8], 40);
                    H1_filters[3] = L *
FIR_filtering(H1_buffer[1], H10, filter_buffer[4], 40);
                    H2_filters[3] = L * FIR_filtering(temp,
H20, filter_buffer[0], 40);
                    break;
            }
        }
        ofstream outfile2("outputM_cos_8_p5.bin", ios::binary
| ios::out);
        outfile2.write(reinterpret_cast<const char*>(output_M),
(n * L / M) * sizeof(double));
        outfile2.close();

        f = 1.0/4;
        output_counter_L = 0;
        output_counter_M = 0;
        m_trck = 0;
        H0_output = 0;
        H1_output = 0;
        H2_output = 0;
        for(int i = 0; i < 2; i++)
        {
            H1_buffer[i] = 0;
        }
```

```
    for(int i = 0; i < 3; i++)
    {
        H0_buffer[i] = 0;
    }
    for(int i = 0; i < n; i++)
    {
        double temp = cos(2 * PI * f * i);
        for(int j = 2; j > 0; j--)
        {
            H0_buffer[j] = H0_buffer[j-1];
        }
        H0_buffer[0] = temp;
        H1_buffer[1] = H1_buffer[0];
        H1_buffer[0] = temp;

        if(i % 4 == 0 )
        {
            for(int j = 0; j < 4; j++)
            {
                H0_output += H0_filters[j];
                H1_output += H1_filters[j];
                H2_output += H2_filters[j];
            }
            output_M[output_counter_M++] = H0_output;
            output_M[output_counter_M++] = H1_output;
            output_M[output_counter_M++] = H2_output;
            H0_output = 0;
            H1_output = 0;
            H2_output = 0;
        }

        switch(i%4) {
            case 0:
```

```
                    H0_filters[0] = L *
FIR_filtering(H0_buffer[2], H03, filter_buffer[11], 40);
                    H1_filters[0] = L *
FIR_filtering(H1_buffer[1], H13, filter_buffer[7], 40);
                    H2_filters[0] = L * FIR_filtering(temp,
H23, filter_buffer[3], 40);
                    break;
                case 1:
                    H0_filters[1] = L *
FIR_filtering(H0_buffer[2], H02, filter_buffer[10], 40);
                    H1_filters[1] = L *
FIR_filtering(H1_buffer[1], H12, filter_buffer[6], 40);
                    H2_filters[1] = L * FIR_filtering(temp,
H22, filter_buffer[2], 40);
                    break;
                case 2:
                    H0_filters[2] = L *
FIR_filtering(H0_buffer[2], H01, filter_buffer[9], 40);
                    H1_filters[2] = L *
FIR_filtering(H1_buffer[1], H11, filter_buffer[5], 40);
                    H2_filters[2] = L * FIR_filtering(temp,
H21, filter_buffer[1], 40);
                    break;
                case 3:
                    H0_filters[3] = L *
FIR_filtering(H0_buffer[2], H00, filter_buffer[8], 40);
                    H1_filters[3] = L *
FIR_filtering(H1_buffer[1], H10, filter_buffer[4], 40);
                    H2_filters[3] = L * FIR_filtering(temp,
H20, filter_buffer[0], 40);
                    break;
            }
        }
```

```
    ofstream outfile1("outputM_cos_4_p5.bin", ios::binary
| ios::out);
    outfile1.write(reinterpret_cast<const char*>(output_M),
(n * L / M) * sizeof(double));
    outfile1.close();

}

void problem4(double* h){
    int n = 100 * N;
    int conv_L = n;
    double f = 1.0/16;
    double output_L[L];
    double output_M[n * L / M];
    double* filter_buffer = new double[N];

    for(int i = 0; i < L; i++)
    {
        output_L[i] = 0;
    }

    for(int i = 0; i < (conv_L + 1)/ M; i++)
    {
        output_M[i] = 0;
    }
    int output_counter_L = 0;
    int output_counter_M = 0;
    int m_trck = 0;

    for(int i = 0; i < n; i++)
    {
        double temp = cos(2 * PI * f * i);
        upsample(temp, L, output_L);
```

```
          for(int j = 0; j < L; j++)
          {
               double filter_output =
FIR_filtering(output_L[j], h, filter_buffer, N);
               if( m_trck % M == 0)
               {
                    output_M[output_counter_M++] =
filter_output;
               }
               m_trck++;
          }
     }

     ofstream outfile1("outputM_cos_16.bin", ios::binary |
ios::out);
    outfile1.write(reinterpret_cast<const char*>(output_M),
(conv_L / M) * sizeof(double));
    outfile1.close();

     f = 1.0/8.0;
     for(int i = 0; i < (conv_L + 1)/ M; i++)
     {
          output_M[i] = 0;
     }
     for(int i = 0; i < N; i++)
     {
          filter_buffer[i] = 0;
     }
     output_counter_L = 0;
     output_counter_M = 0;
     m_trck = 0;

     for(int i = 0; i < n; i++)
```

```
    {
        double temp = cos(2 * PI * f * i);
        upsample(temp, L, output_L);
        for(int j = 0; j < L; j++)
        {
            double filter_output =
FIR_filtering(output_L[j], h, filter_buffer, N);
            if( m_trck % M == 0)
            {
                output_M[output_counter_M++] =
filter_output;
            }
            m_trck++;
        }
    }

    ofstream outfile2("outputM_cos_8.bin", ios::binary |
ios::out);
    outfile2.write(reinterpret_cast<const char*>(output_M),
(conv_L / M) * sizeof(double));
    outfile2.close();

    f = 1.0/4.0;
    for(int i = 0; i < (conv_L + 1)/ M; i++)
    {
        output_M[i] = 0;
    }
    for(int i = 0; i < N; i++)
    {
        filter_buffer[i] = 0;
    }
    output_counter_L = 0;
    output_counter_M = 0;
```

```
    m_trck = 0;

    for(int i = 0; i < n; i++)
    {
        double temp = cos(2 * PI * f * i);
        upsample(temp, L, output_L);
        for(int j = 0; j < L; j++)
        {
            double filter_output =
FIR_filtering(output_L[j], h, filter_buffer, N);
            if( m_trck % M == 0)
            {
                output_M[output_counter_M++] =
filter_output;
            }
            m_trck++;
        }
    }

    ofstream outfile3("outputM_cos_4.bin", ios::binary |
ios::out);
    outfile3.write(reinterpret_cast<const char*>(output_M),
(conv_L / M) * sizeof(double));
    outfile3.close();

}

void problem6(double* h)
{
    ifstream file("audio.bin", ios::binary);
    file.seekg(0, ios::end);
    streamsize fileSize = file.tellg();
    file.seekg(0, ios::beg);
```

```
    size_t numdoubles = fileSize/ sizeof(double);

    double* audio = new double[numdoubles];
    file.read(reinterpret_cast<char*>(audio), fileSize);

    double* output_L = new double[L];
    double* filter_buffer = new double[N];
    int output_counter_M = 0;
    int m_trck = 0;
    double* output_M = new double[numdoubles * L / M];

    auto start = chrono::high_resolution_clock::now();
    for(int i = 0; i < numdoubles; i++)
    {
        double temp = audio[i];
        upsample(temp, L, output_L);
        for(int j = 0; j < L; j++)
        {
            double filter_output =
FIR_filtering(output_L[j], h, filter_buffer, N);
            if( m_trck % M == 0)
            {
                output_M[output_counter_M++] =
filter_output;
            }
            m_trck++;
        }
    }
    auto end = chrono::high_resolution_clock::now();

    auto duration =
chrono::duration_cast<chrono::milliseconds>(end - start);
    cout << "Traditional decimation: " << duration.count()
```

```
<<" ms"<< endl;

    ofstream outfile1("problem6_trad.bin", ios::binary |
ios::out);
    outfile1.write(reinterpret_cast<const char*>(output_M),
(numdoubles * L / M) * sizeof(double));
    outfile1.close();

    double* H0 = new double[160];
    double* H1 = new double[160];
    double* H2 = new double[160];
    double* H00 = new double[40];
    double* H01 = new double[40];
    double* H02 = new double[40];
    double* H03 = new double[40];
    double* H10 = new double[40];
    double* H11 = new double[40];
    double* H12 = new double[40];
    double* H13 = new double[40];
    double* H20 = new double[40];
    double* H21 = new double[40];
    double* H22 = new double[40];
    double* H23 = new double[40];
    int h_tracker = 0;

    split_filter(h, H0, N, 3, 0);
    split_filter(h, H1, N, 3, 1);
    split_filter(h, H2, N, 3, 2);

    split_filter(H0, H00, 160, 4, 0);
    split_filter(H0, H01, 160, 4, 1);
    split_filter(H0, H02, 160, 4, 2);
    split_filter(H0, H03, 160, 4, 3);
```

```
split_filter(H1, H10, 160, 4, 0);
split_filter(H1, H11, 160, 4, 1);
split_filter(H1, H12, 160, 4, 2);
split_filter(H1, H13, 160, 4, 3);

split_filter(H2, H20, 160, 4, 0);
split_filter(H2, H21, 160, 4, 1);
split_filter(H2, H22, 160, 4, 2);
split_filter(H2, H23, 160, 4, 3);

int n = 100* N;
double** filter_buffer_p = new double*[12];


for(int i = 0; i < 12; i++)
{
     filter_buffer_p[i] = new double[40];
}

int output_counter_L = 0;
output_counter_M = 0;
m_trck = 0;

double H0_buffer[3];
for(int i = 0; i < 3; i++)
{
     H0_buffer[i] = 0;
}
double H1_buffer[2];
for(int i = 0; i < 2; i++)
{
     H1_buffer[i] = 0;
```

```
    }
    double* H0_filters = new double[4];
    double* H1_filters = new double[4];
    double* H2_filters = new double[4];

    for(int j = 0; j < 4; j++)
    {
         H0_filters[j] = 0;
         H1_filters[j] = 0;
         H2_filters[j] = 0;
    }

    double H0_output = 0;
    double H1_output = 0;
    double H2_output = 0;

    for(int i = 0; i < 2; i++)
    {
         H1_buffer[i] = 0;
    }
    for(int i = 0; i < 3; i++)
    {
         H0_buffer[i] = 0;
    }

    start = chrono::high_resolution_clock::now();
    for(int i = 0; i < numdoubles; i++)
    {
         double temp = audio[i];
         for(int j = 2; j > 0; j--)
         {
              H0_buffer[j] = H0_buffer[j-1];
         }
```

```
          H0_buffer[0] = temp;
          H1_buffer[1] = H1_buffer[0];
          H1_buffer[0] = temp;
          if(i % 4 == 0 )
          {
               for(int j = 0; j < 4; j++)
               {
                    H0_output += H0_filters[j];
                    H1_output += H1_filters[j];
                    H2_output += H2_filters[j];
               }
               output_M[output_counter_M++] = H0_output;
               output_M[output_counter_M++] = H1_output;
               output_M[output_counter_M++] = H2_output;
               H0_output = 0;
               H1_output = 0;
               H2_output = 0;
          }
          switch(i%4) {
               case 0:
                    H0_filters[0] = L *
FIR_filtering(H0_buffer[2], H03, filter_buffer_p[11], 40);
                    H1_filters[0] = L *
FIR_filtering(H1_buffer[1], H13, filter_buffer_p[7], 40);
                    H2_filters[0] = L * FIR_filtering(temp,
H23, filter_buffer_p[3], 40);
                    break;
               case 1:
                    H0_filters[1] = L *
FIR_filtering(H0_buffer[2], H02, filter_buffer_p[10], 40);
                    H1_filters[1] = L *
FIR_filtering(H1_buffer[1], H12, filter_buffer_p[6], 40);
                    H2_filters[1] = L * FIR_filtering(temp,
```

```
H22, filter_buffer_p[2], 40);
                    break;
                case 2:
                    H0_filters[2] = L *
FIR_filtering(H0_buffer[2], H01, filter_buffer_p[9], 40);
                    H1_filters[2] = L *
FIR_filtering(H1_buffer[1], H11, filter_buffer_p[5], 40);
                    H2_filters[2] = L * FIR_filtering(temp,
H21, filter_buffer_p[1], 40);
                    break;
                case 3:
                    H0_filters[3] = L *
FIR_filtering(H0_buffer[2], H00, filter_buffer_p[8], 40);
                    H1_filters[3] = L *
FIR_filtering(H1_buffer[1], H10, filter_buffer_p[4], 40);
                    H2_filters[3] = L * FIR_filtering(temp,
H20, filter_buffer_p[0], 40);
                    break;
            }
        }
        end = chrono::high_resolution_clock::now();
        duration =
chrono::duration_cast<chrono::milliseconds>(end - start);
        cout << "Polyphase decimation: " << duration.count()
<<" ms"<< endl;
        ofstream outfile3("problem6_poly.bin", ios::binary |
ios::out);
    outfile3.write(reinterpret_cast<const char*>(output_M),
(numdoubles * L / M) * sizeof(double));
    outfile3.close();
}
```